

Tiny Keyword Spotting under Noise and Resource Constraints

Group 21:

Tăng Khôi Nguyên - 2410727

Trần Minh Tuấn - 2411009

Lê Minh Quang - 2410814

Nguyễn Hữu Hoàng Minh - 2410622

Trịnh Hữu Minh - 2410654

Bùi Tuấn Nguyên - 2410716

Lê Đức Minh - 2410588

1. Abstract

Keyword spotting (KWS) enables always-on voice interfaces on microcontroller-class devices, where memory, compute and background noise are severely limited. This study investigates how two compact convolutional networks, a Depthwise Separable CNN (DS-CNN, 23,050 parameters) and BC-ResNet-1 (9,166 parameters), trade-off accuracy, noise robustness and inference cost on ten commands of Google Speech Commands v0.02. We use the official speaker-independent splits, remove 350 clipped training recordings, extract 40-band log-Mel features, and train with random time shifts and background-noise mixing at -5 to 20 dB SNR. Evaluation covers clean speech and 30 noisy conditions built from four seen and two unseen noise sources, averaged over three seeds. Both models reach about 97.2-97.3% clean test accuracy and about 92% under seen noise; on unseen noise DS-CNN scores 90.96% versus 90.11% for BC-ResNet-1, a difference within seed variability. BC-ResNet-1 needs 2.5x fewer parameters and 8.7x fewer MACs, yet DS-CNN is 2.5x faster on a single CPU thread. SpecAugment did not improve robustness. Accuracy falls sharply at -5 dB SNR (about 80-83%).

2. Introduction and Research question

Voice assistants rely on a small always-on detector that listens for a few command words and wakes larger systems only when needed. Such detectors run on battery-powered microcontrollers with tens to hundreds of kilobytes of memory, so a KWS model must be small and cheap while remaining accurate in real acoustic environments. Real environments are the difficult part: kitchens, gyms and streets add background noise that lowers the signal-to-noise ratio (SNR) and shifts the spectral patterns on which the classifier relies. A model that is accurate on clean studio-like recordings may fail in the field, and a model that has only seen a few noise types in training may fail on noises it has never encountered.

This project studies that tension on the ten-command task yes, no, up, down, left, right, on, off, stop, go. We compare two well-known TinyML architectures under one identical, carefully controlled pipeline. The research question is:

RQ: Under a fixed training recipe, how do a Depthwise Separable CNN and BC-ResNet-1 compare in clean accuracy, robustness to seen and unseen background noise across SNR levels, and inference cost (parameters, MACs, activation memory, CPU latency)?

We derive four testable hypotheses:

H1 Both models exceed 96% clean test accuracy

H2 Accuracy drops more on unseen than on seen noise

H3 The model with fewer parameters/MACs is also faster on CPU

H4 SpecAugment improves noise robustness. The contributions are (i) a leakage-aware protocol with disjoint speakers and disjoint noise segments, (ii) an explicit seen/unseen noise test with 30 deterministic conditions, (iii) a multi-seed paired comparison, and (iv) a cost analysis that includes measured latency rather than MACs alone.

3. Related work

Small-footprint KWS: Early neural KWS replaced HMM-based systems with deep neural networks that run at low cost (Chen et al., 2014). Convolutional networks followed, offering better accuracy with fewer parameters by exploiting local time-frequency structure (Sainath & Parada, 2015), and residual networks improved accuracy further on Speech Commands (Tang & Lin, 2018).

Architectures for microcontrollers: Hello Edge benchmarked DNN, CNN, RNN and DS-CNN families under microcontroller memory and compute budgets and found DS-CNN the best accuracy-per-cost option (Zhang et al., 2017). DS-CNN factorizes a convolution into a per-channel depthwise filter and a 1x1 pointwise filter, an idea popularized by MobileNets (Howard et al., 2017). More recently, Broadcasted Residual Learning (BC-ResNet) processes frequency with cheap 1D depthwise operations, averages over frequency, applies a temporal convolution once, and broadcasts the result back, reaching high accuracy with under 10k parameters in its

smallest form (Kim et al., 2021). Sub-spectral normalization, used inside BC-ResNet, normalizes groups of frequency bands separately (Chang et al., 2021).

Data, augmentation and benchmarks: The Speech Commands dataset standardized evaluation of limited-vocabulary recognition (Warden, 2018). SpecAugment masks frequency bands and time frames of the spectrogram and is widely used as regularization (Park et al., 2019). Multi-condition training, i.e. mixing noise into training audio at random SNRs, is the standard way to obtain noise robustness. MLPerf Tiny provides a benchmark for tiny-device inference (Banbury et al., 2021).

Gap: Papers usually report clean accuracy, parameters and MACs. Fewer report a controlled comparison where some noise sources are withheld entirely from training and where measured CPU latency is placed next to MAC counts. This project fills that gap at small scale.

4. Dataset and Data preparation

Dataset: Google Speech Commands v0.02 (Warden, 2018): 105,829 one-second utterances of 35 words from 2,618 speakers, released under CC BY 4.0.

- Download: http://download.tensorflow.org/data/speech_commands_v0.02.tar.gz
- Catalog page: https://www.tensorflow.org/datasets/catalog/speech_commands
- Paper: <https://arxiv.org/abs/1804.03209>

(The version is identified from the file counts: the ten target words total 38,546 recordings, which matches v0.02, and `_background_noise_` contains the six standard noise files.)

Task and splits: Only the ten commands are used (class IDs 0-9). The official `validation_list.txt` and `testing_list.txt` assign each file to a split; all other files form the training set. The code asserts that the speaker sets of the three splits are pairwise disjoint (speaker ID is the file-name prefix before `_nohash_`), that the lists do not overlap, and that every class occurs in every split.

Cleaning: Microphone saturation distorts spectral features. Every training file is audited on raw 16-bit samples, and a file is dropped when more than 0.5% of its samples have $|\text{amplitude}| \geq 32,440$ (≈ 0.99 full scale). Integer arithmetic is done in int32 to avoid overflow at -32768. Validation and test sets are not filtered, so evaluation stays comparable to the official protocol.

Split	Recordings	Note
Raw train	30,769	official complement of val/test
Clean train	30,419	350 clipped files removed (1.1%)
Validation	3,703	official list
Test	4,074	official list

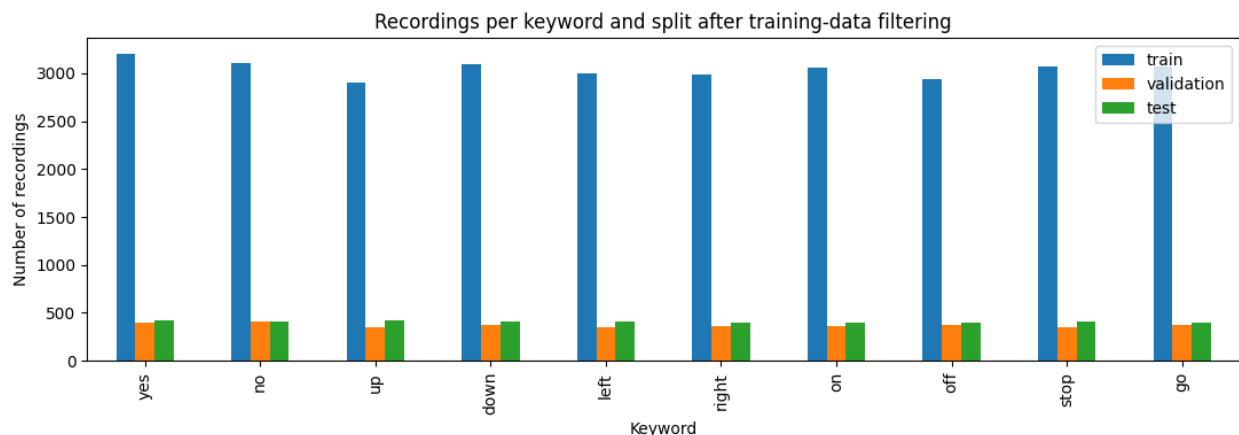


Figure 1: Class distribution across training, validation, and test splits for the ten target keywords after microphone clipping filtering.

The share removed per class ranges from 0.71% (yes) to 1.49% (up), so class balance is preserved (about 2,900-3,200 clean training clips per class). After cleaning, the mean duration is 0.982 s; 2,825 clips (9.3%) are shorter than one second (minimum 0.299 s), the mean RMS level is -25.4 dBFS (std 6.1 dB) and the mean peak amplitude is 0.53.

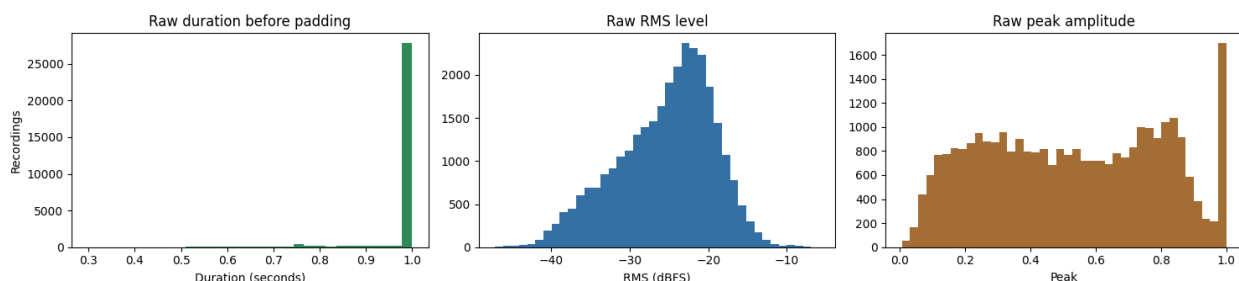


Figure 2: Exploratory distributions of raw training recordings prior to zero-padding: (a) clip duration showing 9.3% shorter utterances, (b) root-mean-square (RMS) energy distribution, and (c) peak amplitude distribution.

Waveform preparation: All files are validated as mono, 16 kHz, PCM16. Clips shorter than 16,000 samples are center-padded with zeros; clips longer than one second raise an error.

Background noise: The six files of `_background_noise_` are split by source, not at random:

Role	Sources	Use
Seen	doing_the_dishes, exercise_bike, pink_noise, white_noise	first 80% train, next 10% validation, last 10% seen-test
Unseen	running_tap, dude_miaowing	complete files, final test only

Seen segments are therefore disjoint in time (e.g. doing_the_dishes: 76.1s/ 9.5s/ 9.5s; exercise_bike: 49.0s/ 6.1s/ 6.1s; pink_noise and white_noise: 48s/ 6s/ 6s). The unseen files (running_tap, dude_miaowing) last 61.156s and 61.8s.

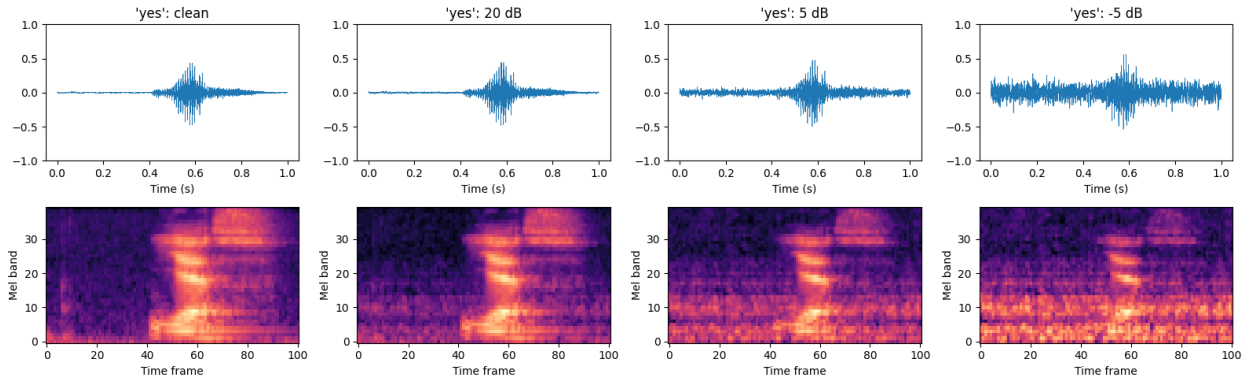


Figure 3: Waveforms (top) and corresponding 40-band normalized log-Mel spectrograms (bottom) for a sample utterance under clean speech and background noise (doing_the_dishes) mixed at 20 dB, 5 dB, and -5 dB SNR.

5. Methods

5.1 Common pipeline

Front-end: A GPU module converts a waveform batch (B, 16000) into log-Mel features (B, 1, 40, 101): 30 ms window (480 samples), 10 ms hop (160 samples), 512-point FFT, 40 HTK Mel bands, power spectrum, centered STFT with reflection padding, conversion to dB with an 80 dB dynamic-range floor, then per-clip mean/variance normalization.

Training augmentation (applied on the waveform): (i) A random time shift of ± 100 ms with zero fill; (ii) with probability 0.8, a random one-second crop from the training noise bank is added at an SNR drawn uniformly from $[-5, 20]$ dB. The noise gain is computed from signal powers so that the exact SNR is achieved; a common-gain peak limiter then keeps $|x| \leq 1$ without changing the SNR. Nearly silent noise crops are rejected and redrawn. SpecAugment (one frequency mask of at most 6 bands and one time mask of at most 10 frames) exists as an optional layer used only in the ablation.

5.2 Baseline method: DS-CNN

Following the DS-CNN topology of Hello Edge: a 4x10 convolution (stride 2, 64 channels) with BN and ReLU, then four blocks of depthwise 3x3 $\rightarrow$ BN $\rightarrow$ ReLU $\rightarrow$ pointwise 1x1 $\rightarrow$ BN $\rightarrow$ ReLU, global average pooling and a linear layer. It has 23,050 parameters (90 KB in FP32) and 21.67 M MACs. Its operations are dense 1x1 convolutions that map well to optimized CPU kernels.

5.3 Main method: BC-ResNet-1

A broadcasted-residual network with width factor $\tau = 1$: a 5x5 head convolution (stride 2 along frequency, 16 channels) followed by four stages with [2, 2, 4, 4] blocks and channels [8, 12, 16, 20]. Each block has:

- (a) a frequency-wise depthwise 3x1 convolution with sub-spectral normalization (5 groups);
- (b) averaging over the frequency axis;
- (c) a temporal depthwise 1x3 convolution with dilation 1, 2, 4, 8 by stage, BN, SiLU, a 1x1 convolution and Dropout2d(0.1);
- (d) the temporal result is broadcast back over frequency and added to the frequency-wise output plus an identity shortcut. The first block of stages 2 and 3 halves the frequency axis. The classifier is a depthwise 5x5 convolution, a 1x1 expansion to 32 channels, global pooling and a 1x1 output layer. It has 9,166 parameters (35.8 KB) and 2.48M MACs.

5.4 Comparison strategy

Both models share one recipe, one front-end, the same data, the same three seeds (2026, 2027, 2028) and the same data order and augmentation per seed, which allows paired differences by seed. Reported quantities are: clean accuracy; accuracy averaged over seen-noise conditions and over unseen-noise conditions; accuracy per SNR and per noise type; parameters, FP32 size, MACs, largest single-layer activation (a lower bound on activation RAM) and CPU latency. Results are given as mean $\pm$ standard deviation over seeds.

6. Experimental setup

Training recipe (identical for all runs): SGD with momentum 0.9, weight decay $1e-3$, batch size 100 (305 steps per epoch), 100 epochs, cross-entropy loss. The learning rate rises linearly to 0.1 over 5 warm-up epochs and then follows a per-step cosine decay to zero. After each epoch the model is scored on clean validation and on noisy validation (validation noise bank, random noise and a random SNR in {20, 10, 5, 0, -5} dB per clip, fixed per index); the checkpoint with the highest mean of the two scores is kept. Training ran on a CUDA GPU (about 21-30 minutes per run).

Setup 1: Baseline vs main model: DS-CNN and BC-ResNet-1 are trained with three seeds (6 runs) and compared on validation accuracy, clean test accuracy and size/MACs.

Setup 2: Main research experiment (noise robustness): Every checkpoint is evaluated on the clean test set and on 30 noisy test conditions = 6 noise sources (4 seen test segments, 2 unseen full recordings) x 5 SNRs (20, 10, 5, 0, -5 dB). Noise is mixed deterministically per clip index, so all models see identical inputs. Results are aggregated into seen (20 conditions) and unseen (10 conditions) groups, by SNR and by noise type.

Setup 3: Analysis, ablation and cost: (a) Ablation: the same recipe with SpecAugment switched on, three seeds per model, compared with the paired no SpecAugment runs. (b) Inference cost: single CPU thread, batch size 1, 30 warm-up and 300 timed runs per measurement, eager PyTorch and frozen TorchScript with BN folded; the log-Mel front-end is timed separately.

Sanity checks run before training: All 38,196 clips load and are non-silent; 100 real noise mixes (plus synthetic edge cases) reach the requested SNR within 10^{-3} dB, before and after peak limiting; noisy validation datasets are deterministic; SpecAugment returns unmasked features in evaluation mode; batch shapes and label ranges are correct.

7. Results and Discussion

7.1 Setup 1: baseline vs main model

Model	Params	Size (FP32)	MACs	Val clean (%)	Val noisy (%)	Test clean (%)
BC-ResNet-1	9,166	35.8 KB	2.48 M	96.82 ± 0.09	91.92 ± 0.37	97.30 ± 0.20
DS-CNN	23,050	90.0 KB	21.67 M	96.69 ± 0.07	91.59 ± 0.19	97.20 ± 0.15

Both models exceed 96% clean accuracy (H1 supported). The clean gap is 0.10 ± 0.05 percentage points (pp). To judge its size, note that with 4,074 test clips one clip is 0.025 pp and the binomial standard error at 97% accuracy is about 0.27 pp, so a 0.1 pp gap is not meaningful. In practice the two models are equivalent on clean speech, and BC-ResNet-1 achieves this with 2.5x fewer parameters and 8.7x fewer MACs.

7.2 Setup 2: robustness to seen and unseen noise

Test accuracy (%), mean and std over seeds

Model	Clean	Seen noise	Unseen noise
BC-ResNet-1	97.30 ± 0.20	91.98 ± 0.11	90.11 ± 1.14
DS-CNN	97.20 ± 0.15	91.88 ± 0.30	90.96 ± 0.20
Paired diff. (BC - DS), pp	$+0.10 \pm 0.05$	$+0.10 \pm 0.29$	-0.86 ± 1.04

Accuracy by SNR (mean over noise types and seeds, %):

Model	Group	20 dB	10 dB	5 dB	0 dB	-5 dB
BC-ResNet-1	seen	96.80	95.67	94.10	90.75	82.58
BC-ResNet-1	unseen	96.25	94.44	92.21	88.12	79.52
DS-CNN	seen	96.61	95.51	93.87	90.59	82.82
DS-CNN	unseen	96.07	94.76	92.98	89.61	81.39

Accuracy by noise type (mean over SNRs and seeds, %):

Model	doing_the_dishes	exercise_bike	pink_noise	white_noise	running_tap	dude_miaowing
BC-ResNet-1	90.13	90.72	94.23	92.85	90.53	89.69
DS-CNN	89.31	91.16	94.18	92.88	92.33	89.60

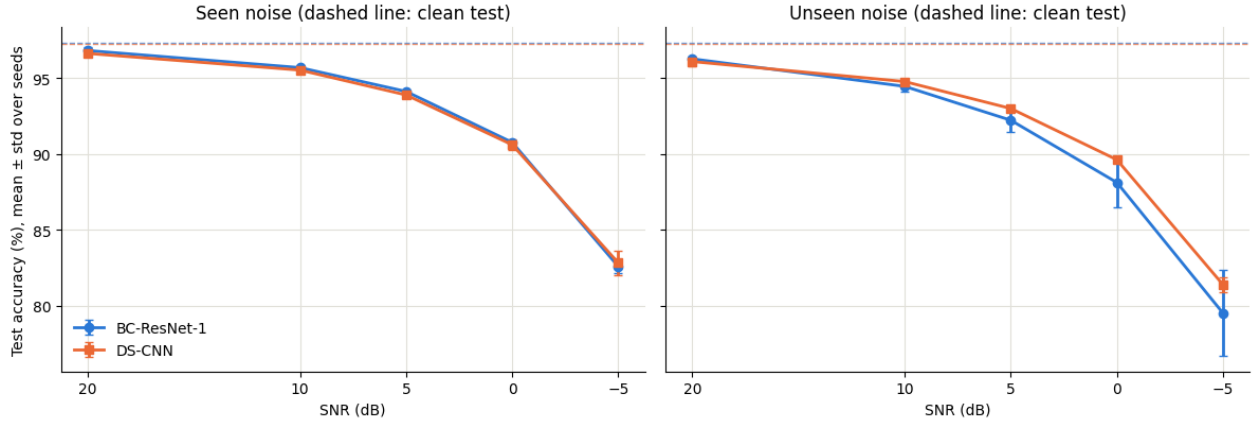


Figure 4: Test accuracy versus SNR for BC-ResNet-1 and DS-CNN under seen (left) and unseen (right) background noise.

Discussion: (1) Noise costs about 5 pp on seen sources for both models (97.3 $\rightarrow$ 92.0 and 97.2 $\rightarrow$ 91.9), which is the price of evaluating down to -5 dB. (2) Unseen noise is harder (H2 supported): the drop from clean is 7.2 pp for BC-ResNet-1 and 6.2 pp for DS-CNN, versus 5.3 pp on seen noise. The gap is concentrated at low SNR: at -5 dB, BC-ResNet-1 loses 3.1 pp when moving from seen to unseen noise, DS-CNN loses 1.4 pp. (3) The ranking on unseen noise should not be over-interpreted. DS-CNN is ahead by 0.86 pp on average, but the paired difference varies between +0.28 and -1.77 pp across seeds (std 1.04), and BC-ResNet-1's unseen accuracy has a seed std of 1.14 against 0.20 for DS-CNN. With three seeds this is a hint, not evidence, that DS-CNN generalizes more consistently. (4) The source-level pattern is informative: the gain of DS-CNN is almost entirely on `running_tap` (92.33 vs 90.53); on `dude_miaowing` the models are tied. Stationary broadband noises (pink, white) are the easiest, while the non-stationary sources are the hardest.

7.3 Setup 3a: inference cost

Model	MACs (M)	Largest activation (KB)	Eager (ms)	TorchScript (ms)	Front-end (ms)	End-to-end (ms)
BC-ResNet-1	2.48	126.3	3.91	3.80	0.33	4.24
DS-CNN	21.67	255.0	1.38	0.96	0.33	1.71

DS-CNN costs 2.5x the parameters, 8.7x the MACs and 2.0x the peak activation memory of BC-ResNet-1, yet runs in 0.40x the time. H3 is therefore rejected for latency: fewer MACs did not mean faster inference on this CPU. A likely explanation, not tested here, is that BC-ResNet-1 consists of many small depthwise, dilated and reshape/averaging operations with low arithmetic intensity and high per-operator overhead, whereas DS-CNN spends its MACs in dense 1x1 convolutions that CPU libraries execute efficiently. The practical consequence is that the better model depends on the target: on a microcontroller whose flash and RAM are the binding constraint, BC-ResNet-1 is favored (36 KB of weights, half the activation memory); on a CPU where latency dominates, DS-CNN is favored. Both models spend under 5 ms per second of audio, far below real-time limits, and the front-end adds only 0.33 ms.

7.4 Setup 3b: SpecAugment ablation (test accuracy, %, mean $\pm$ std over 3 seeds)

Model	SpecAug	Clean	Seen	Unseen
BC-ResNet-1	off	97.30 $\pm$ 0.20	91.98 $\pm$ 0.11	90.11 $\pm$ 1.14
BC-ResNet-1	on	97.05 $\pm$ 0.09	91.60 $\pm$ 0.28	89.23 $\pm$ 0.87
DS-CNN	off	97.20 $\pm$ 0.15	91.88 $\pm$ 0.30	90.96 $\pm$ 0.20
DS-CNN	on	97.41 $\pm$ 0.38	91.94 $\pm$ 0.27	89.94 $\pm$ 0.85

Paired differences (on - off, pp): BC-ResNet-1 -0.25/ -0.38/ -0.88 and DS-CNN +0.20/ +0.06/ -1.03 for clean/ seen/ unseen. H4 is not supported: SpecAugment does not help on unseen noise and its effects on clean and seen noise are within one standard deviation. A plausible reason is that these networks have only 9-23k parameters, so they are capacity-limited rather than over-fitted. Training accuracy at epoch 100 with SpecAugment is only about 91% (training data are already corrupted at -5 dB), so extra masking mainly makes the task harder for a small model. The curves also show that validation accuracy is unstable while the learning rate is near its 0.1 peak (e.g. BC-ResNet-1, seed 2026: 85.5% at epoch 20, 90.0% at epoch 30, 82.6% at epoch 40) and stabilizes only as the cosine schedule anneals, so the final epochs matter most.

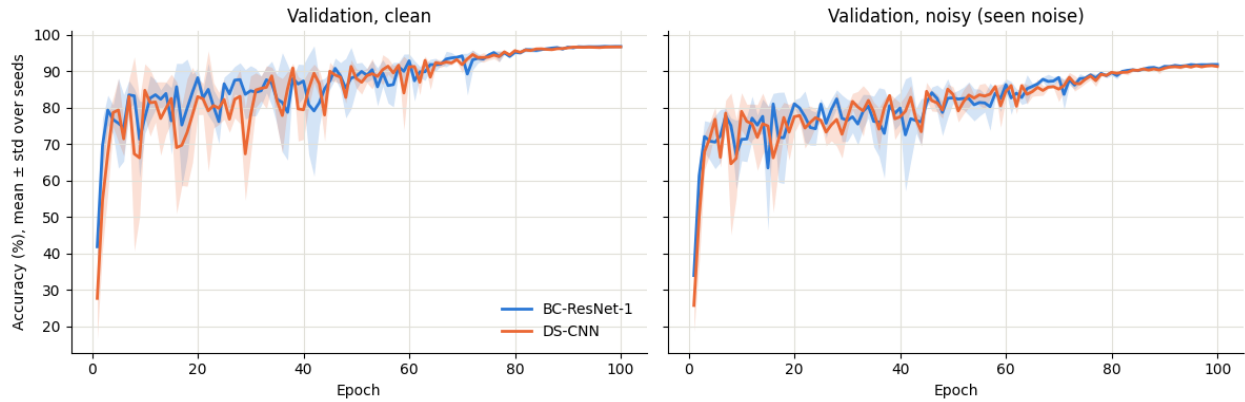


Figure 5: Validation accuracy trajectories over 100 epochs on clean validation and seen noisy validation.

8. Error and Qualitative analysis

Error cases observed in the results:

- 1. Very low SNR:** The largest error mass is at -5 dB, where error is 17.2-20.5% (accuracy 79.5-82.8%), compared with 3.2-3.9% at 20 dB. Each 5 dB step below 5 dB adds roughly 3-9 pp of error. Reason: at -5 dB the noise has about three times the power of the speech, so short, low-energy consonants (the cues that distinguish no/go, up/off) are masked in the Mel spectrogram, and per-clip normalization cannot restore information that the noise has overwritten.
- 2. Non-stationary noise:** doing_the_dishes (about 10% error for both models) and dude_miaowing (about 10.3%) are the hardest; pink noise (5.8%) is the easiest. Reason: pink and white noise are stationary with smooth spectra, and they were present in training, so the network can learn to discount them. Dishes contain impulsive clatter that resembles plosive bursts, and meowing is harmonic with energy in the vowel-formant region, so both imitate speech cues that the classifier uses.
- 3. Generalization to unseen sources:** Unseen noise adds a further 1-2 pp of error beyond seen noise, and for BC-ResNet-1 the outcome varies strongly by seed (paired difference from +0.28 to -1.77 pp). Reason: with only four training noise sources, the learned invariance is source-specific; sub-spectral normalization and frequency averaging in BC-ResNet-1 may make its behavior depend more on the particular optimization trajectory (a hypothesis not tested here).
- 4. Data-level failure modes:** 350 clipped training files (1.1%, highest for up at 1.49%) were removed, so the models never see saturated speech and may be less robust to it at test time, because the test set is not filtered. 9.3% of clips are shorter than one second (as short as 0.3 s) and are zero-padded, so the amount of silence varies by clip and truncated words are possible.

5. **Training instability:** Validation accuracy swings by more than 10 pp between epochs at high learning rate, so a single "best epoch" can be partly luck; the multi-seed average mitigates this.

Clean Test Confusion Analysis: The empirical confusion matrix on clean test speech confirms the phonetic confusion hypotheses. The top 5 misclassifications concentrate on words sharing acoustic and formant characteristics:

1. **off** → **up** (12 errors): Shared short open central vowels with weak unvoiced plosive/fricative offsets.
2. **go** → **no** (10 errors): Shared rhyming diphthong/ou/, where the initial voiced velar stop/g/ and nasal/n/ are hard to separate.
3. **off** → **on** (8 errors) & **up** → **off** (7 errors): Close vowel heights with reduced consonant energy.
4. **go** → **down** (8 errors): Confused back vowel transitions. Per-keyword recalls remain high across all classes: yes (99.8%), stop (99.8%), right (99.0%), no (98.3%), left (98.3%), on (97.0%), up (96.9%), down (96.8%), go (94.5%) and off (94.5%).

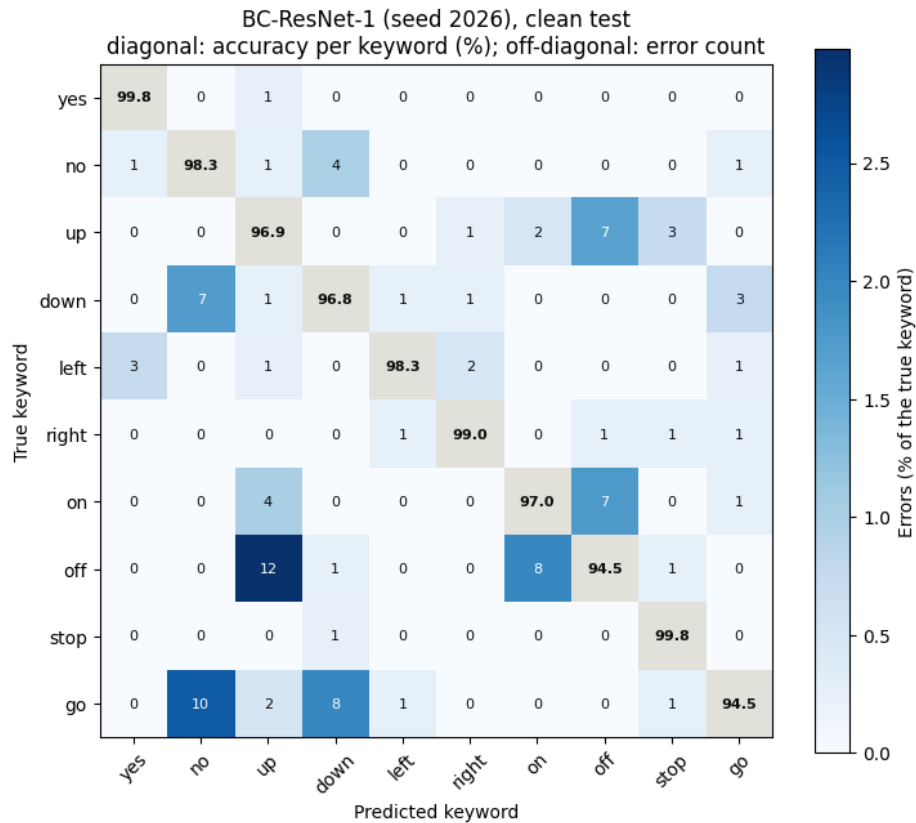


Figure 6: Confusion matrix of BC-ResNet-1 (seed 2026) on the clean test set (4,074 clips), showing per-class recall along the diagonal and error counts on off-diagonal pairs.

Real-world microphone validation: To test deployment beyond the benchmark dataset, we recorded one second of speech ("yes") directly from a microphone, kept it in memory as a WAV clip, and mixed it with out-of-domain hand-clapping noise (my_noise.wav) at all five SNR levels. Both models predict "yes" correctly from clean speech down to +5 dB SNR (BC-ResNet-1: 81.9% confidence; DS-CNN: 58.6%). When noise power matches or exceeds speech power, performance degrades: at 0 dB BC-ResNet-1 still predicts "yes" but with only 40.5% confidence, while DS-CNN misclassifies it as "off" (51.5%); at -5 dB both models fail (BC-ResNet-1: "left", 13.5%; DS-CNN: "off", 35.7%). This mirrors the steep accuracy drop below 5 dB in the benchmark results.

9. Conclusion and Limitations

Conclusion: Under an identical recipe, a 9k-parameter BC-ResNet-1 and a 23k-parameter DS-CNN both reach about 97.2-97.3% clean accuracy, about 92% under seen noise and about 90-91% under unseen noise, with no statistically convincing accuracy difference. BC-ResNet-1 wins on parameters, MACs and activation memory; DS-CNN wins on CPU latency by 2.5x. Noise robustness is limited mostly by very low SNR (-5 dB) and by non-stationary sources, unseen noise costs about 1-2 pp more than seen noise, and SpecAugment gave no benefit for these tiny networks. The choice between the models therefore depends on whether memory or latency is the deployment bottleneck.

Limitations:

- Only three seeds; most seed-paired differences are smaller than their standard deviation, so no significance test is claimed.
- Only two unseen noise sources, with additive noise only (no reverberation, microphone mismatch or distance), so "unseen" results do not represent all real environments. Seen test segments are only 6-9.5 s long, and the same clean test speech is reused in all 30 conditions, so conditions are not independent.
- Closed-set task: ten keywords with no "unknown" or "silence" class, which is easier than a deployed detector that must reject non-keywords and false alarms.
- Cost is measured on a laptop CPU with FP32 weights, not on a microcontroller; MACs count only convolution and linear layers, and the largest activation is a lower bound on RAM. INT8 quantization, which is standard on microcontrollers, is not evaluated.
- Hyper-parameters were not tuned per model, and model selection used seen-noise validation data, which may slightly favor seen-noise results.

Confusion analysis is evaluated on clean test speech, full multi-condition confusion evaluation across all 30 noise conditions remains future work.

Future work: multi-condition confusion analysis under noise, more seeds and noise sources, INT8 quantization and on-device latency, added "unknown/silence" classes, and noise-aware training objectives.

References:

1. Warden, P. (2018). Speech Commands: A Dataset for Limited-Vocabulary Speech Recognition. arXiv:1804.03209.
2. Zhang, Y., Suda, N., Lai, L., & Chandra, V. (2017). Hello Edge: Keyword Spotting on Microcontrollers. arXiv:1711.07128.
3. Kim, B., Chang, S., Lee, J., & Sung, D. (2021). Broadcasted Residual Learning for Efficient Keyword Spotting. In INTERSPEECH 2021. arXiv:2106.04140.
4. Chang, S., Park, H., Cho, J., Park, H., Yun, S., & Hwang, K. (2021). Subspectral Normalization for Neural Audio Data Processing.
5. Howard, A. G., et al. (2017). MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications. arXiv:1704.04861.
6. Park, D. S., et al. (2019). SpecAugment: A Simple Data Augmentation Method for Automatic Speech Recognition. In INTERSPEECH 2019. arXiv:1904.08779.
7. Chen, G., Parada, C., & Heigold, G. (2014). Small-footprint Keyword Spotting Using Deep Neural Networks.
8. Sainath, T. N., & Parada, C. (2015). Convolutional Neural Networks for Small-footprint Keyword Spotting.
9. Tang, R., & Lin, J. (2018). Deep Residual Learning for Small-footprint Keyword Spotting.
10. Banbury, C., et al. (2021). MLPerf Tiny Benchmark.